

# Parte III: Processamento de Linguagem Natural para Linguistas

## Unidade 4 - Respostas dos Exercícios

CiberExt 26-29 · FEELT38103 · Universidade Federal de Uberlândia

Agosto de 2026

### Respostas — Introdução aos *word embeddings*

#### Parte A — Conceitos

##### Resposta 1.

A hipótese distribucional afirma que **semelhança de significado resulta em semelhança de distribuição linguística**: palavras que ocorrem em contextos semelhantes tendem a ter significados semelhantes.

A citação de Firth a antecipa ao deslocar o critério de identificação do significado da palavra isolada para sua **companhia**, isto é, para os elementos com que ela coocorre. Firth não formula a hipótese em termos distribucionais quantitativos — isso é feito por Harris (1954) —, mas estabelece o princípio metodológico de que o significado é contextual e, portanto, observável no uso.

##### Resposta 2.

- O eixo **sintagmático** é o eixo da **combinação**: descreve como os elementos escolhidos se encadeiam horizontalmente para formar estruturas maiores (*Helsinki + is + the capital of + Finland*).
- O eixo **paradigmático** é o eixo da **seleção**: descreve o conjunto de alternativas que poderiam ocupar uma mesma posição na estrutura (*Helsinki / Tallinn / Estocolmo*). As alternativas se definem por oposição.

Na matriz da seção 2.1:

- Cada **coluna** corresponde a um lema único, isto é, a uma **escolha paradigmática** disponível.
- Cada **linha** corresponde a um *Doc*, isto é, a uma **estrutura sintagmática** concreta, que registra quais escolhas paradigmáticas foram efetivamente combinadas.

##### Resposta 3.

A representação é chamada de “saco de palavras” porque registra apenas **quais** palavras ocorrem e **quantas vezes**, descartando completamente a informação sobre a **ordem** em que aparecem. É como se as palavras fossem jogadas em um saco e agitadas.

Duas limitações:

1. **Perda da ordem e da estrutura.** As orações *o cão mordeu o homem* e *o homem mordeu o cão* recebem exatamente o mesmo vetor, embora signifiquem coisas opostas.
2. **Crescimento da dimensionalidade e esparsidade.** Cada palavra nova do vocabulário exige uma dimensão adicional. Como cada documento contém apenas uma fração ínfima do vocabulário, a esmagadora maioria das dimensões vale zero, o que desperdiça memória e dificulta o cálculo de distâncias significativas.

#### Resposta 4.

Um vetor **esparso** é aquele em que a maior parte das dimensões vale zero e apenas poucas carregam informação — como o vetor one-hot, em que exatamente uma dimensão vale 1. Um vetor **denso** é aquele em que praticamente todas as dimensões têm valores não nulos, cada uma codificando alguma parcela de informação.

Os embeddings da camada oculta são densos porque a rede é **forçada** a comprimir a informação de 24 dimensões esparsas em apenas 2 dimensões. Como não há espaço para reservar uma dimensão por palavra, cada dimensão precisa passar a codificar informação distribuída sobre a palavra e seu contexto.

#### Resposta 5.

O objetivo final não é dispor de um preditor de palavras vizinhas — essa predição, isoladamente, tem pouca utilidade prática. O objetivo é obter **representações numéricas úteis** das palavras.

A predição funciona como tarefa substituta porque **só é possível resolvê-la bem se a rede aprender boas representações**. Para prever que *the* e *capital* aparecem perto de *be*, a rede precisa codificar, nos pesos da camada oculta, informação sobre o comportamento distribucional de *be*. Ao fim do treinamento, descartamos o preditor e ficamos com os pesos — que são os embeddings.

Trata-se do princípio geral do **aprendizado auto-supervisionado**: o rótulo é extraído do próprio texto, sem anotação humana.

#### Resposta 6.

A camada oculta é um gargalo porque é muito mais estreita (2 neurônios) do que as camadas de entrada e de saída (24 unidades cada). Toda a informação necessária para a predição precisa **passar por essas duas dimensões**.

Esse estrangulamento é justamente o que produz o aprendizado. Se a camada oculta tivesse 24 neurônios, a rede poderia simplesmente copiar a

entrada, memorizando cada palavra individualmente, sem generalizar nada. Ao ser obrigada a comprimir, a rede é forçada a **agrupar palavras com comportamento distribucional semelhante** na mesma região do espaço — que é exatamente o que a hipótese distribucional prevê.

## Parte B — Programação

### Resposta 7.

```
from sklearn.metrics.pairwise import cosine_similarity

# Compara os vetores dos Docs nos índices 3 e 4
cosine_similarity([df.values[3]], [df.values[4]])

array([[0.37796447]])
```

O valor é coerente. As orações são:

Travelling between Helsinki and Tallinn takes about two hours  
Ferries depart from downtown Helsinki and Tallinn

Elas compartilham três lemas — Helsinki, and e Tallinn —, o que produz uma similaridade moderada (0,38). Esse valor é claramente inferior ao dos *Docs* 0 e 1 (0,67), que compartilham quatro lemas em uma estrutura sintagmática idêntica (*is the capital of*), mas superior ao dos pares que quase nada compartilham.

### Resposta 8.

```
# Acrescenta a nova oração à lista
examples.append("Stockholm is the capital of Sweden")

# Reprocessa toda a lista
docs = list(nlp.pipe(examples))

# Reconta os lemas
lemma_counts = {i: doc.count_by(LEMMA) for i, doc in enumerate(docs)}
lemma_counts = {i: {docs[i].vocab[k].text: v for k, v in counter.items()}
                 for i, counter in lemma_counts.items()}

# Reconstrói o DataFrame
df = pd.DataFrame.from_dict(lemma_counts).sort_index(ascending=True).fillna(0).T

# Verifica o formato
print(df.shape)

# Recalcula a matriz de similaridade
sim = cosine_similarity(df.values)
print(sim[5])

(6, 26)
```

```
[0.66666667 0.66666667 0.40824829 0.          0.          1.          ]
```

O *DataFrame* passa a ter **26 colunas**: as 24 originais mais os dois lemas novos, Stockholm e Sweden.

Os *Docs* 0 e 1 tornam-se **igualmente** os mais similares ao novo *Doc* 5, ambos com 0,67. Isso faz sentido: as três orações compartilham exatamente a mesma estrutura sintagmática (*X is the capital of Y*), diferindo apenas nos dois nomes próprios. É precisamente a previsão da hipótese distribucional: Helsinki, Tallinn e Stockholm são alternativas paradigmáticas de uma mesma posição.

#### Resposta 9.

```
# Similaridade entre 'capital' e 'the'
print(cosine_similarity([lemma_df['capital'].values], [lemma_df['the'].values]))

# Similaridade entre 'Helsinki' e 'ferry'
print(cosine_similarity([lemma_df['Helsinki'].values], [lemma_df['ferry'].values]))

array([[0.53846154]])
array([[0.13363062]])
```

**Comentário.** O par *capital* / *the* apresenta similaridade alta (0,54) porque ambos ocorrem repetidamente nos mesmos contextos locais (*the capital of*), compartilhando vizinhos como *be*, *of* e um *ao outro*. Isso ilustra um ponto importante: a hipótese distribucional captura **similaridade de distribuição**, que nem sempre corresponde a similaridade de significado — um determinante e um substantivo não são sinônimos.

O par *Helsinki* / *ferry* apresenta similaridade baixa (0,13): as duas palavras ocorrem em vizinhanças bastante distintas no nosso pequeno corpus.

#### Resposta 10.

```
# Recria o DataFrame vazio
lemma_df_3 = pd.DataFrame(index=unique_lemmas, columns=unique_lemmas).fillna(0)

# Percorre os Tokens, agora com janela de três palavras de cada lado
for token in combined_docs:

    # Note o intervalo ampliado de posições
    for position in [-3, -2, -1, 1, 2, 3]:

        try:
            nbor_lemma = token.nbor(position).lemma_
            lemma_df_3.at[token.lemma_, nbor_lemma] += 1

        except IndexError:
            continue
```

```
# Recalcula a similaridade
cosine_similarity([lemma_df_3['Tallinn'].values], [lemma_df_3['Helsinki'].values])
array([[0.56577895]])
```

**Comparação.** Com janela 2, a similaridade era de **0,43**; com janela 3, sobe para **0,57**. Ampliar a janela aumenta o número de vizinhos compartilhados registrados e, portanto, o número de dimensões não nulas em comum, elevando a similaridade.

De modo geral, janelas menores tendem a capturar relações mais **sintáticas** (colocações imediatas), enquanto janelas maiores capturam relações mais **temáticas** ou tópicas. A escolha da janela é, portanto, uma decisão analítica, não um detalhe técnico.

#### Resposta 11.

```
import numpy as np

def one_hot(lemma, vocabulary):
    """Devolve o vetor one-hot de um lema em relação a um vocabulário.

    Parâmetros:
        lemma: string com o lema a ser codificado
        vocabulary: lista de lemas únicos

    Devolve:
        Um array NumPy do tamanho do vocabulário.
    """

    # Verifica se o lema pertence ao vocabulário
    if lemma not in vocabulary:

        # Levanta um erro informativo caso contrário
        raise ValueError(f"O lema '{lemma}' não está no vocabulário.")

    # Cria um vetor de zeros com o tamanho do vocabulário
    vector = np.zeros(shape=len(vocabulary))

    # Define como 1 a posição correspondente ao lema
    vector[vocabulary.index(lemma)] = 1

    return vector

# Testes
print(one_hot('Helsinki', unique_lemmas))
print(one_hot('Curitiba', unique_lemmas))
```

```
[0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
ValueError: O lema 'Curitiba' não está no vocabulário.
```

### Resposta 12.

```
# Reconstrói a rede com quatro neurônios na camada oculta
network_input = keras.Input(shape=targets.shape[1], name='input_layer')

# Apenas o argumento 'units' muda
hidden_layer = Dense(units=4, activation=None, name='hidden_layer')(network_input)

output_layer = Dense(units=context.shape[1], activation='softmax',
                      name='output_layer')(hidden_layer)

model_4d = keras.Model(inputs=network_input, outputs=output_layer)
model_4d.compile(loss='categorical_crossentropy')

model_4d.fit(x=targets, y=context, batch_size=64, epochs=1500, verbose=0)

# Verifica o formato dos pesos da camada oculta
weights_4d = model_4d.get_weights()[0]
print(weights_4d.shape)

(24, 4)
```

A matriz de pesos passa a ter formato (24, 4): cada um dos 24 lemas é agora representado por um vetor de **quatro** dimensões.

Já não é possível plotar o espaço diretamente porque um gráfico de dispersão só dispõe de dois eixos, e teríamos quatro coordenadas por ponto. Para visualizar, seria necessário aplicar uma técnica de **redução de dimensionalidade** — como PCA ou t-SNE — que projeta os vetores de quatro dimensões em duas, preservando aproximadamente as distâncias relativas. Essa é exatamente a abordagem adotada na próxima unidade, ao visualizar os embeddings do spaCy, que têm centenas de dimensões.

### Resposta 13.

```
# Localiza os índices dos dois lemas
i_hel = unique_lemmas.index('Helsinki')
i_tal = unique_lemmas.index('Tallinn')

# Compara os embeddings aprendidos
cosine_similarity([hidden_layer_weights[i_hel]], [hidden_layer_weights[i_tal]])

array([[0.60469307]])
```

**Comparação.** A similaridade calculada a partir da matriz de coocorrência era de **0,43**; a partir dos embeddings aprendidos, obtemos aproximadamente **0,60**.

Duas ressalvas importantes:

1. O valor exato **varia entre execuções**, pois os pesos são inicializados aleatoriamente. O que se deve observar é a tendência, não o número.
2. Em apenas duas dimensões, quaisquer dois vetores tendem a apresentar similaridade de cossenos relativamente alta, simplesmente porque há pouco espaço para que fiquem distantes. Comparar valores absolutos entre espaços de dimensionalidades diferentes é, portanto, enganoso.

#### Resposta 14.

1. **Vocabulário insuficiente para generalizar.** Com 24 lemas e 118 pares, quase todas as palavras ocorrem uma ou duas vezes. A rede não tem evidência estatística suficiente para distinguir regularidade de acaso: uma única coocorrência tem o mesmo peso que um padrão sistemático.
2. **Sobreajuste ao exemplo específico.** O modelo aprende as cinco orações particulares, não a língua inglesa. **Helsinki** e **Tallinn** ficam próximos porque aparecem literalmente lado a lado nos dados, e não porque o modelo tenha inferido que ambos são capitais bálticas.
3. **Ausência de palavras funcionais em contexto variado.** Palavras como **the** e **of** ocorrem em pouquíssimos contextos distintos, quando na realidade sua característica definidora é ocorrer em uma enorme variedade deles. Suas representações ficam, portanto, distorcidas — e, como são frequentes, distorcem também as representações das palavras vizinhas.

Uma quarta consequência, mais sutil: as próprias métricas de avaliação tornam-se pouco informativas, já que não há dados retidos para teste.

#### Resposta 15.

O modelo é incapaz de fazer a distinção porque atribui **exatamente um vetor a cada forma lexical**. O dicionário `lemma_vectors` mapeia a *string* `'banco'` a um único vetor one-hot, e a camada oculta produz um único embedding correspondente. As duas acepções de “banco” são, para o modelo, literalmente a mesma entrada — e o vetor resultante acaba sendo uma média confusa dos dois contextos de uso.

Esse é o limite dos chamados embeddings **estáticos** (Word2vec, GloVe, fastText): um tipo lexical, um vetor.

Para superá-lo é necessário produzir embeddings **contextuais**, em que a representação de uma palavra depende da sentença em que ela ocorre. Isso exige:

- Uma arquitetura capaz de **ler a sentença inteira** ao codificar cada palavra — tipicamente um *Transformer* com mecanismo de atenção, em vez de uma rede que recebe uma única palavra isolada como entrada.
- Uma tarefa de treinamento que **force o uso do contexto**, como prever uma palavra mascarada a partir de toda a sentença (o objetivo de *masked language modelling* usado pelo BERT).

- Volume muito maior de dados e de parâmetros, para que os diferentes contextos de uso de cada forma sejam efetivamente observados.

Nesses modelos, as ocorrências de “banco” em *sentei no banco* e em *fui ao banco* recebem vetores distintos, pois cada representação é computada em função das palavras que a cercam. Os embeddings contextuais obtidos de *Transformers* são o tema da próxima unidade.